EcoMods Booklet

The EcoMods, Würzburg

2026-11-11

Contents

1 About this book **5**

1.1 Acknowldegements and licensing 5

1.2 Contributing to this book 5

1.3 Missing sections and TODOs within this booklet 6

1.4 People 6

1.5 Induction 6

1.6 What you can find in this booklet 7

2 Programming tools **9**

2.1 Basic tools and languages 9

2.2 Advanced tools 13

2.3 Best practices 14

3 IT infrastructure **17**

3.1 SSH 17

3.2 Using HPC Julia 20

3.3 Using slurm 22

3.4 Using CCTB Server = gaias and planets 23

3.5 X2GO 24

4 Process-based models **25**

4.1 EcoMods models 25

5 Computational methods **27**

5.1 Sensitivity Analyses 27
5.2 Virtual ecologist 28
5.3 Parameter optimization 28
5.4 Machine Learning 29
5.5 Phylogenetic Analyses 29
5.6 Functional Analyses 29
5.7 SDMs 29

6 Project management **33**

6.1 Computational Notebooks 33
6.2 Excel-based orga 33
6.3 Pomodoro timer 34
6.4 Asana 34
6.5 Finishing a PhD 34

7 Writing & Publishing **35**

7.1 Tools 35
7.2 Writing Papers 35
7.3 Rebuttal Letters 38
7.4 Reviewing Papers 38

Chapter 1

About this book

Hello and welcome to the EcoMods working group!

In this document, you will (hopefully) find everything you need for your course / internship / thesis with us.

The book is meant to be a live document, and will be updated continuously. As you browse through sections, you will encounter empty or incomplete sections. You're welcome to ask for sections to be added, updated, or even better, improve it yourself! See section **??**.

1.1 Acknowldegements and licensing

This manual is an update of a manual of the original Ecomods at the University of Würzburg. Huge thanks to them for assembling this knowledge base!

The manual is published under the terms of the MIT License, which means you can freely copy, modify and share its contents as long as you acknowledge the original work and provide a copyright notice and copy in the license in all derived works.

1.2 Contributing to this book

We gladly accept any suggestions on how to improve this guide, so let us know if you feel that anything is missing!

This booklet is a bookdown document. This means, that it consists of several R Markdown files. To learn R markdown see section 2.1.1.4. The booklet is hosted on GitHub (see section 2.1.3.

1. Step: Clone the repository from GitHub.
2. Step: Open the .rmd file of the Chapter you would like to change.
3. Step: Do your changes.
4. Step: Commit your changes and push them on GitHub.
5. Step: Then the magic happens: via github actions the website will be updated, but this takes 5-10 Minutes (check the green dot!). This creates files that are pushed automatically. Therefore, don't work within the Repo while waiting and **pull the changes** before you go on editing.

Don't forget to add links, citations, figures etc in a nice and organized way. Check text for typos and readability and be sure that a beginner can understand it.

1.3 Missing sections and TODOs within this booklet

If you find any missing topics, collect them here or write a *TODO* at the appropriate part.

1.4 People

[TODO]

1.5 Induction

Welcome to the group! We hope you have a great time with us and that your experience in your lab will be the opportunity to develop your academic skills and explore exciting research!

Here are some things you should consider upon joining us:

- **Keys**: our lab is hosted in the old NEES institute, Venusbergweg 22, at the edge of the Botanische Gärten. Access to the building and other facilities of the institute is managed through transponders, which you must obtain in person from Elke Friesecke at the main building of the Nees-Institut (Meckenheimer Allee 170). You will need to return the transponders upon leaving.

- **Slack**: we have a Slack workspace for the group, which is our main channel for communication. You will find links to the lab meeting schedule and zoom call pinned in the #general channel. Ask another team member for an invite link!
- **Whatsapp**: we have a Whatsapp convo for instant communication (mostly socials). Ask a team member for an invite if you'd like to join.
- **Calendar**: We keep a Google Calendar to pin and share relevant events. If you'd like to get access, send a google account address to Theo.
- **Meetings**: save these time slots for regular meetings:
  - Weekly Ecomod lab meeting, Mondays 17:00-18:00.
  - NEES seminar (jointly with Prof. Max Weigend's group) Tuesdays 16:30-17:30

The schedule for the lab meetings is available here. It is also possible to join online, the zoom link is pinned in the Slack's general channel.

- **IT questions:** Theo can help with all research-related software enquiries. For hardware issues, the responsible person for BIOB is Tomislav Balen. In particular, for security reasons our printer can only be accessed with specific drivers, which are to be installed by Tomi.
- **University Library:** before using the services of the ULB (Universitäts und Landesbibliothek Bonn), you will need to get yourself a library ID card (Bibliothekausweis) and a user account, which are not automatically created upon joining the University. The procedure is detailed on the ULB website. You will need to submit a form, get the Uni Bonn app and after a few days you will receive instructions on how to link your Uni-ID into the app. Finally, you will need to head to the main ULB building to show a piece of ID, proof of residence and proof of affiliation with Uni Bonn (enrolment certificate or work contract).

1.6 What you can find in this booklet

You'll find information about the following topics:

- **Programming tools** in Chapter 2
- **IT infrastructure** in Chapter 3
- **Process based models** in Chapter 4
- **Computational methods** in Chapter 5

- **Project management** in Chapter 6
- **Writing & Publishing** in Chapter 7

Chapter 2

Programming tools

2.1 Basic tools and languages

The most commonly used programming languages in our lab are R and Julia. They are both great languages for biology-related programming and statistical analysis.

If you already know how to program and are looking for a challenge, our lab members enjoy participating in Advent of Code. It forces you to think outside the box and will teach you some new tricks!

2.1.1 R

R is very well-established and probably has the most support from all the languages we use. Most of the time, you can copy-paste your error message from R into Google, and you will find several solutions for your problem. A very helpful website is Stack Overflow.

2.1.1.1 Interface

In our lab, most people use RStudio when they are working in R. RStudio allows you to very easily write and execute your scripts, create plots and keep an eye on your environment. It also includes a helpful debugging mode. Other, not as commonly used GUIs are RKWard, BlueSky and R Commander.

2.1.1.2 Resources

To get started, there are several books on how to use R.

R for Data Science: "You'll learn how to get your data into R, get it into the most useful structure, transform it, visualise it and model it."

Advanced R: "The book is designed primarily for R users who want to improve their programming skills and understanding of the language. It should also be useful for programmers coming to R from other languages, as help you to understand why R works the way it does."

Further reading:

- Coding Etiquette (in R) - slides from *Anne* prepared for BioInfo F2 2020 (ask Anne)
- Troubleshooting in R - Slides from *Anne* prepared for BioInfo F2 2020 (ask Anne)
- R for Dummies, Andrie de Vries & Joris Meys, 2015 (PDF available, ask Jana)

2.1.1.3 Packages

What are R packages?

If you have a problem that you need a certain function to solve, many times you do not need to write that function from scratch yourself. Usually, another (more skilled) person has already written some code that solves this problem. These "packages of code" can be uploaded to CRAN (the Comprehensive R Archive Network) and then downloaded to be used by other users.

An example: You have a nice plot and want to ensure it is easy to read and interpret for colorblind people. Now you could set all of your 200 colors by hand so that it is not the default red-green nightmare, but it is much easier to just download a package called "viridis" that contains multiple color palettes that are optimized for this very use. Then, all you need to do is tell R to use the viridis palette instead of setting all colors by hand, which would get very tedious. Installing and loading a package via CRAN:

```
install.packages("PackageName")
library(PackageName)
```

Note that packages need to be loaded into your environment using the `library()` command in order for R to use them. You can also download packages that are not yet on CRAN using devtools.

Commonly used packages in our working group include (but are not limited to):

- tidyverse

- here
- devtools
- ggplot2
- viridis
- raster
- rgdal

2.1.1.4 Rmd

R Markdown is a great way to create documents in HTML, Word or PDF format. Rmds can include chunks of code which create the tables and plots of the document. This way, you don't have to update your document parallel to your code, as you keep both your code to create your plots and said plots in the file. This file was also created using Rmd! You can also use .Rmd files to create computational notebooks.

Best overall guide is linked.

A future alternative might be quarto.

2.1.2 Julia, the language

Julia is a relatively new programming language with a significantly higher performance than R. For complex models that take a lot of time to run in R, it might make sense to transfer them to Julia. While it has many advantages, it is not as well-known as R, so you may need to search for an explanation for your error message a bit longer. Julia has its own Discourse page with many helpful entries and tutorials as well.

The Julia Documentation is also VERY good.

To quickly look up Julia syntax, you can use this Cheat Sheet.

2.1.2.1 Best tutorials

To get started, here is a very nice book for beginners by Ben Lauwens and Allen Downey that explains not only the Julia language, but also introduces the concept of programming in general.

JuliaAcademy offers an introductory course that is tried and liked by Robin.

If you are not that into reading, the official Julia Programming Language Youtube channel has some very nice tutorials structured by playlists. One

especially helpful playlist is Julia Programming for Nervous Beginners. They also hold the annual JuliaCon online and upload the speaker's talks to Youtube if you cannot be there live.

2.1.2.2 Plotting in Julia

Plots in Julia can be a bit more tricky than in R (but that might be because we are used to plotting in R - don't let this discourage you from plotting your heart out in Julia if that is what you want to do!). Here are some packages that may help facilitate your plotting adventures:

- This page shows a list of graphical plotting packages
- GLMakie is used by *Robin*

2.1.2.3 Interface

As for R, you can choose from a variety of interfaces for Julia. In our working group, these are the most common ones:

- Atom and Juno for julia. Here is a nice tutorial on how to set it up. However, even though popular in our lab, Juno will receive no more feature updates as of March 2022, so if you are new to Julia, it might be best to choose a different IDE.
- VSCode is the successor of Juno. It works in a similar way and is also used by some members of our lab and is at this point the recommended option to use.
- if you are interested in using a Computational Notebook (see section below), Jupyter might be a good fit for you. Learn how to use Jupyter and Julia together here. This is still a bit buggy, though.

2.1.3 Version control

Git is a great software that allows you to keep track of the changes you make in your code (version control). This way, you will not need to keep several files named "Code_02_revised" and "Code_02_revised_changed" etc, which can quickly get confusing. There are different servers that you can store your repositories on, the most common ones being GitHub and GitLab. You can install git on your PC and use it via the commandline, or you can use it with the help of a UI, like GitHub Desktop. This is a very helpful Training and Tutorial that explains what git is and how to use it very nicely.

2.1.4 Bash commands

When we let our models run, they tend to need a lot of computational power. Running experiments on your laptop can take days, weeks or even months. Therefore, we can send our scripts to more powerful machines from the University infrastructure. To tell the computer "execute script XXX" you need to write a short shell using bash.

A nice tool to use for bash scripts is shellcheck - Robin

For an example bash script see section IT infrastructure - Using HPC Julia

2.2 Advanced tools

2.2.1 Singularity

Singularity is a way to use containers, small virtual machines that use the resources of the computer they run on but have their own environment. This will only be a short explanation on how to use them otherwise refer to the official documentation.

Before a container can be used it first has to be built. This can be done by either using a `.def` file (Only if you have sudo rights on the local machine and under Linux so use Linux or WSL) or you can get it on the base of an existing container in the Singularity library or from docker. Prebuilt containers on the gaias are stored in `/storage/singularity_images`. `ecomodsdm.sif` is a container based on `docker://rocker/geospatial`

The main ways to use Singularity containers are with the commands:
- - `singularity shell [container.sif]` opens a shell in the provided container - `singularity exec [container.sif]` This executes a predefined command as defined in the definition file - `singularity exec [container.sif] [command]` This only executes the requested command e.g. opens R or executes a program
By default `/home/$USER` is bound into the container at execution. If more folders need to be bound into it do this with the `--bind [local]:[container]` option

Here is a users guide.

2.2.2 EMACS, Vim, and Spacemacs

TODO

2.2.3 Regex

Robin

- A handy website that goes through the basics of Regular Expression grammar.
- WHAT IS THAT? https://regex101.com/

2.2.4 Backup and Storage of Data and Code

For code: via git, e.g. on GitHub

For data?

2.3 Best practices

2.3.1 Best Modeling and Programming practices

There are some practices in modelling and programming that make the job a lot easier. Our former colleague, Daniel, has put together some thoughts on what to keep in mind:

1. Use relative paths
2. Document your code and how to use it Using Github/Gitlab. Gitlab is self-hosted (Biology). Zenodo is citable, trusted third-party.
    1. User Manual
    2. DeveloperManual
3. Script (automatize) experimental steps (i.e. pipeline)
    1. Analyses in a single R script
4. Use version control
    1. Use commit identifier to refer to a particular code version for a experiment
    2. Use seeds for replicate (must be robust for changes in OS)
    3. Map and config file as output (for seed for example) Data longevity & accessibility: Use storage folder in gaias. Physical backup, repository of very large data. Container for codes that require dependencies and particular versions.

2.3.2 Blog post series on software development

Daniel also has a nice blog post series if you want to read more.

1. Principles of Software Development
2. Understandable Software
3. The Art of Abstraction
4. Dealing with Errors
5. Programming Tools: Languages
6. Programming Tools: Paradigms
7. Developing in a Team
8. Final Thoughts

See also the paper from Daniel [Vedder et al., 2021].

Chapter 3

IT infrastructure

See the CCTB WueCampus course for documentation of our computing tools and tutorials for common software.

In general, there are two powerful computers that you can run your code on - the Julia HPC cluster and the gaias. The gaias are the owned by the CCTB, while the Julia HPC belongs to the faculties of Biology, Physics and Chemistry and is maintained by the Rechenzentrum. See below on how to connect to the respective servers.

3.1 SSH

3.1.1 Connect with SSH

There's several ways to connect directly to the servers. Apple and Linux have inbuilt ssh clients in the terminal. From Windows 10 onward powershell also comes with an ssh client, though be aware it does have some hiccups. For simple tasks PuTTy is sufficient. If you're trying something more complicated (ssh tunnel) you could try git bash or get Windows Subsystem for Linux

3.1.2 Using SSH to authenticate your Github account TODO: move to Github section OR set links

SSH stands for secure shell protocol. It allows you to securely connect to a remote computer using a commandline interface. Please note that you must be connected to the university network (e.g. via VPN) to access our servers.

3.1.2.1 Step 1: Generate an SSH keypair on the server and copy public key to Github

TODO what github docs should be checked? For details, check Github Docs. Following instructions are adapted from here. Log in to the server (e.g. Curta). Check if you have existing ssh keys with

```
ls -al ~/.ssh
```

Generate a new ssh key:

```
ssh-keygen -t ed25519 -C "your_email@example.com
```

Make sure that you use the email that you also used for your Github account. Accept the default file name by pressing Enter key and type a password when prompted. You can also leave the password blank if you don't want to protect your SSH key with an additional password.

Start your ssh agent:

```
eval "$(ssh-agent -s)"
```

Add your new ssh key to ssh agent. If your key has a different name, replace id_ed25510 with the respective name of your key

```
ssh-add -K ~/.ssh/id_ed25519
```

Add your new ssh key to your Github account (the public key): First, copy the public ssh key with (or any other method of copying of your choice):

```
cat ~/.ssh/id_ed25519.pub
```

Now the key should be printed to the terminal. Copy the key (it starts with ssh-ed255…. and ends with your email address) to the clipboard. Go to your Github account, click the icon on the upper-right corner > Settings > SSH and GPG Keys > Click New SSH key. Give your new ssh key a name and paste the copied key in the key text area. Then click Add SSH Key and if prompted, type your account's password.

3.1.2.2 Step 2: Test the connection

You can test the connection with

```
ssh -T git@github.com
```

It can be that the host cannot be authorized the first time that you try to connect. If you want to add Github to the known hosts, you can just type yes and the next time you won't see this warning anymore. Now you can clone, push and pull from a repository without having to enter your password and username all the time. Note: This only works if you clone a repository via ssh and not via https!

3.1.3 ssh alias

When using `ssh`, `scp` or `rsync` on Linux to connect to or move files always having to type out a user and IP adress can get tiring. There's a couple ways around that but the easiest is provided by the ssh tool itself. It is possible to configure an alias that replaces the whole user and IP address and can be filled with the autocomplete.

To accomplish this:

1. find the path where your ssh configuration is located. This should be a folder in your user folder called `.ssh` (When using `ls` it only shows you the some of the folders, use `ls -a` to find folders leading with a dot.)
2. move into the folder (`cd ssh`) and create a file named config (eg. `touch config`)
3. in the file the configuration is formatted like the following:

```
HOST hostname1
  SSH_OPTION value
  SSH_OPTION value

HOST hostname2
  SSH_OPTION value

HOST *
  SSH_OPTION value
```

more specific options go to the top, more general ones further down. Everything in Hostname * is for all ssh sessions.

If you were to configure a shortcut for say gaia5 it would look something like this:

```
Host gaia5
  User yourusername
  HostName 132.187.198.19
```

4.Write into the file and save it. You should now be able to use your shortcut with ssh.

for further details you can check online, I found this site also explains it nicely and in more detail:
https://linuxize.com/post/using-the-ssh-config-file/

TODO: how to do a key pair

3.2 Using HPC Julia

Robin - check over it TODO

General information by the Julia support

The Julia HPC Cluster only coincidentally has the same name as the Julia Programming language. You can use it to run your models and also store your data on it if you want. Here is how to use it:

1. The first step to using the Julia HPC Cluster is to get an account. To do this, go here. Simply go to "Benutzer freischalten" at the very bottom of the page and choose "Ecosystem Modeling" from the drop down menu. You should soon get an email about your account being created. Now it's time to get started!

2. Depending on your PC's operating system, you will access the HPC differently.

- on Windows: install PuTTy and WinSCP. WinSCP allows you to transfer data between the cluster and your own PC. With PuTTy, you can access the HPC and tell it what to do. Connect to both via your JMU-Account name like so: name[at]julia.uni-wuerzburg.de and your JMU password. In PuTTy, you will need to use linux commands for the cluster. A linux tutorial can be found here.

- on linux: add how to here

3. You will probably need to install all packages for R and julia that the code you want to run uses. You can do this in R very easily. Type `R` and press enter for R to open. Then, give the command `install.packages("package")` as you usually would. The packages should install without problem. For julia, you might need to install it

first. Instructions can be found here To install the packages, start Julia by typing `julia` and pressing enter. Then, type `using Pkg` and install your packages as usual with `Pkg.add("package")`.

4. You can use your normal scripts that you also use to run your code on your PC. Just double check that there are no absolute path names. Remember to save results! To execute your script, you need a bash file. A good way to create these files is using Notepad++. Open Notepad++ and create a new file (top left). Important: go to Edit, Format end of line and choose UNIX. Otherwise, the cluster will not be able to read your file. In your bash file, your code will look something like this for running an R script:

```
#!/bin/bash
#SBATCH -J #NameOfJob
#SBATCH -c 2 #this is your number of cores to use
#SBATCH --mail-type=ALL #if you want to receive emails when your job starts/fails/finishes, u
#SBATCH --mail-user=name\@uni-wuerzburg.de #(mailto:mail-user=name@uni-wuerzburg.de){.email}
#SBATCH -p test #(where do you want your job to run?)
#SBATCH -t 3-00:00:00 #(how much time is allocated at maximum?)
#SBATCH --mem=100G #(how much memory do you need?)
Rscript "Test.R"
```

5. Run your bash script using “sbatch name.sh”. You can use “squeue” to check if your code is pending (PD) or running (R). Depending on your settings in the bash script, you might get email notifications about your job. When it has finished, you can transfer the results from the cluster to your PC using WinSCP.

Happy coding!

3.2.1 Calculate you CO2 Emission from HPC

Step 1: log into HPC and run the following command

```
sacct --user=<your-user> --starttime=2021-01-01 --format='CPUTimeRAW' > usage_2021.txt
```

Step 2: Download usage_2021.txt from HPC

Step 3: Open usage_2021.txt locally and remove the second row wi

Step 4: Use R to calculate CPU hours

```
usage <- readr::read_csv("usage_2021.txt")
dplyr::summarize(usage, sum_CPU_hours = sum(CPUTimeRAW)/60/60)
```

Step 5: Calculate climate impact

Go to https://green-algorithms.org/ Enter the following for HPC: Runtime: number calculated above Type of cores: CPU Number of cores: 1 (because the hours are CPU hours) Model: Any Memory available: average memory that you used (I put 10 GB) Platform: Local server Select correct location and put no to all additional questions asked about CPU

3.3 Using slurm

Robin TODO
SLURM is the resource manager running on the gaias and the julia log-in node. It's job is to distribute tasks thorough the run-time among the planets so you and others can all use them at the same time. While the julia cluster has a resource manager the planets don't and queue everything directly regardless of how often you've used it so please be considerate.
Some of the most common commands are:
- `srun [command]` gives the following command directly to SLURM and has it output to your current shell. If you get disconnected in the meantime you won't be able to reconnect so be careful - `sbatch [batchscript]` passes a job script to SLURM and puts it in the queue. One of the planets will execute all the steps in the job script and puts all the output in a `.out` file as specified in the options. You can also use an sbatch script to submit several jobs at the same time - `scancel [jobid]` use this to cancel a running job - `squeue` show currently running jobs - `sinfo` show current status of the whole cluster - `sstat [jobid]` give stats of a currently running or recently finished job
Here are some of the most useful options for `srun` and `sbatch`(reminder: you can always check for more options yourself by putting in `srun --help` or search for *man srun*):
- `--job-name` gives your job a name for `squeue` and the like - `--output` where to put the output of your batch file. Default is `slurm-%A_%a.out` where *%A* is the job ID and *%a* is the array index. - `-n --ntasks` how many tasks you want to have run simultaneously. - `-t --time` Maximum time your job will run. If it runs longer it will be killed
- `--mem` request this much memory. Default is in MB, add G to request GB. If you don't specify the whole node will be requested.
- `-w --nodelist` request a specific node or list of nodes, they will *all* be used--a –array`requests a job array` –mail-type`SLURM can send you mails to update you about the status of your jobs. Most common is ALL, check documentation for more.` –mail-user`Set your email here Here's another summary by the people who make SLURM: https://slurm.schedmd.com/pdfs/summary.pdf Helpful commands are:` -srun -w [node] –mem=[needed memory] –pty bash`requests to open a bash run time on the specified node. You can either`

`start R or Julia from here or check how much memory or CPU is used with`htop‘

3.4 Using CCTB Server = gaias and planets

Here at the CCTB our own servers are the gaias and the planets. While the gaias are more powerful than your own PC their main job is to be used for smaller tasks and to serve as log-in nodes to get access to the more powerful planets. running smaller bits of code on the gaias is no problem, but if you have more consider having it run on the planets.

3.4.1 Opening an Rstudio server on the planets

An Rstudio server works basically the same way as Rstudio on your local machine. Only that your code is running somewhere else so you are able to access the power of our computational infrastructure.

1. Find the script `rstudio-server.sh` in internal/scripts and copy it to a directory on one of the gaias you are using.
2. From your chosen directory start the SLURM job with the command `sbatch rstudio-server.sh`
3. SLURM will tell you your job is running and gives you your job ID. Open the log file that was created with your job number (you can use something like `cat` or `less`) `cat rstudio-server.job.[jobID]`
4. Follow the instructions in the job file. These should be to connect via a ssh tunnel (on windows 10 and up you can use powershell, if that doesn't work refer to the ssh section), open your favorite browser and point it to `localhost:8787`where you'll find a log-in screen for your rstudio server. Log-in with the given credentials.
5. When you are done close the server by pressing the off button on the top right and cancel the SLURM job by entering `scancel -f [jobID]`
   Some notes:/

- The way the base script works after 8 hours SLURM will send a termination signal to the r server which will then save everything and next time you open a new session you should again have your environment back
- If you need more memory or a longer run time you can change it in the script header using typical SLURM syntax

- See the section on Singularity and containers (in progress) if you want to change anything about the container. You will probably have to change quite a bit of the script to use a different one than the one that's used
- The way this job script is set up you can't install new R packages. This is so this will work out of the box with no conflicts. You can enable access of the R session to other packages by commenting out line 31 `export R_LIBS_USER=/usr/local/lib/R/library,/usr/local/lib/R/site-library` at your own risk.
- The script is based on Rocker

3.5 X2GO

TODO

Chapter 4

Process-based models

4.1 EcoMods models

Here, you will find all of our working group's models. For specific questions for each model, please refer to the respective lab member.

4.1.1 GeMM

TODO Robin

4.1.2 MGM

MGM (Macrophyte Growth Model) is a process-based, eco-physiological model simulating the growth of submerged macrophytes under different environmental conditions. MGM is a simplified re-implementation of Charisma 2.0 [van Nes et al., 2003] in Julia language. The model scheme is depicted in 4.1 .

Documentation and code is open access on GitHub.

Further questions: ask Anne

4.1.3 metaRange

general info - TODO JANA & STEFAN

4.1.3.1 R version

TODO Stefan

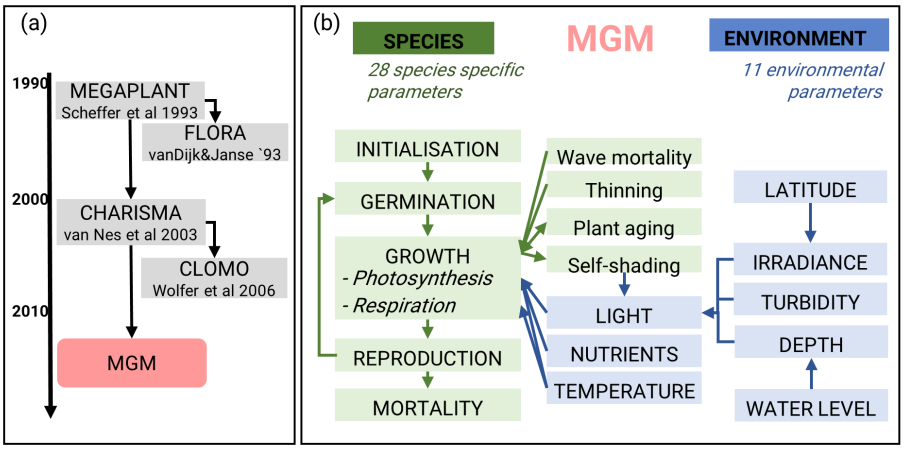


Figure 4.1: Model phylogeny of MGM (a) and simplified model scheme (b).

4.1.3.2 Julia Version

TODO Jana

Chapter 5

Computational methods

Now, let's come to the more practical part of our Beginner's Guide. Here, you will get an overview on the methods that we use in our working group.

5.1 Sensitivity Analyses

Methods that determine how target variables are affected based on changes in other variables known as input variables.

5.1.1 Overview

General broad overview in **Sensitivity Analysis in Practice : A Guide to Assessing Scientific Models** [Saltelli et al., 2002]

Comparison of different sensitivity analysis methods: A performance comparison of sensitivity analysis methods for building energy models Idea: Run the sensitivity analysis not on the data itself but on a comparison between data and model (as you would use in the validation) -> This is the measure to which you want to know the sensitivity

5.1.2 Local

= One-factor-at-a-time (OAT)

TODO Anne

5.1.3 Global

Morris screening and enhancement as used in the R package sensitivity (Campolongo 2007).

TODO Anne

5.2 Virtual ecologist

TODO

5.3 Parameter optimization

5.3.1 Direct methods

Differential evolution optimization Differential Evolution with DEoptim (R package) How does differential evolution work?

TODO Anne

5.3.2 Bayesian Calibration

florianhartig/LearningBayes: An introduction to Bayesian statistics Bayesian Tools - General-Purpose MCMC and SMC Samplers and Tools for Bayesian Statistics Inverse modeling https://arxiv.org/ftp/arxiv/papers/2007/2007.15580.pdf

5.3.3 Emulatoren

Approximieren das Modell selbst “modelling a model” / “meta-model”

TODO

5.3.4 Neural networks

https://arxiv.org/ftp/arxiv/papers/2007/2007.15580.pdf https://link.springer.com/chapter/10.1007%2F978-3-030-44584-3_43 https://royalsocietypublishing.org/doi/10.1098/rsbl.2017.0660

Bayesian https://bg.copernicus.org/articles/15/5801/2018/

TODO

5.3.5 Tips and Tricks

For all methods, first, run an optimization on a dataset that you produced with the model. That way, you can find out whether the optimization algorithm can find the parameters with which you modelled the dataset. This can help to verify that your optimization method works.

5.4 Machine Learning

If you are interested in machine learning, here is a course by our colleague Florian Hartig from the University of Regensburg. This course is held in R.

5.5 Phylogenetic Analyses

TODO

5.6 Functional Analyses

Wright et al (2005) provides a good basis for 'trusting' trait-trait associations above 5-fold variation and/or n=20 spp, within a study [Wright et al., 2005]. Although Wright et al. show trait correlations with as few as 4 species.

For multivariate analyses:

- MANOVA (because traits are not independent from one another) and canonical discriminant analyses (CDA) =Y check R package candisc
- trait network analyses (R package igraph - Csardi and Nepusz, 2006)
  - Mantel test for comparing trait correlations across groups (e.g. taxonomic, trophic or guild groups)
- PCA (useful for visualization as well)

5.7 SDMs

ToDo: WHAT IS A SDM?

TODO: organize better and make references readable

Tutorials:

- Damaris Zurell: https://damariszurell.github.io/SDM-Intro/
- Babak Naimi: https://www.youtube.com/watch?v=83dMS3bcjJM
- https://www.biogeoinformatics.org/
- Daniel Griffith: https://griffithdan.github.io/pages/outreach/SDM-Workshop-OSU-FALL2017.pdf

Further:

- ModestR Software: http://www.ipez.es/ModestR/
- ModestR Tutorial: https://www.slideshare.net/modestrsoftware/21-creating-virtual-species-and-calculating-simple-ensemble-models-with-r-that-can-be-later-used-with-version-modestr-56-or-higher

Earlier models:

- BIOCLIM: https://doi.org/10.1111/ddi.12144

Packages:

- sdm: https://cran.r-project.org/web/packages/sdm/sdm.pdf
- kuenm: https://peerj.com/articles/6281/
- virtualspecies [Leroy et al., 2015]
- https://cran.r-project.org/web/packages/virtualspecies/virtualspecies.pdf

For Maxent type of models:

- Evaluating Maxent niche models with ENMeval: https://doi.org/10.1111/2041-210X.12261

For ensemble models:

- Biomod Paper: https://doi.org/10.1111/j.1600-0587.2008.05742.x
- Biomod2 package:
- https://cran.r-project.org/web/packages/biomod2/biomod2.pdf
- https://github.com/biomodhub/biomod2

- ESDM: https://doi.org/10.1111/2041-210X.13283
- Stacked SDMs (for species richness/diversity):
- SSDM: https://cran.r-project.org/web/packages/SSDM/vignettes/SSDM.html
- Joint-SDMs for co-occurrence data:
- with HMSC: https://doi.org/10.1111/2041-210X.13345
- sjSDM: https://doi.org/10.1111/2041-210X.13687
- https://github.com/TheoreticalEcology/s-jSDM

Useful literature:

- Review about SDMs [Hao et al., 2019]
- ODMAP: A standard protocol for reporting SDMs [Zurell et al., 2020]
- Correcting sampling bias in MAxent [Kramer-Schadt et al., 2013]

Chapter 6

Project management

Apart from all the programming you are going to learn, it might also make sense for you to learn how to structure your work day/week and your whole project so that you never lose track of what you need to do, and what you already have done, and how to do it all efficiently.

6.1 Computational Notebooks

These are files that allow you to write descriptive text, code, and the output of that code. They are particularly useful for:

- learning and teaching coding, because the code and its output are shown together, so you always know what does what
- documenting analysis/model development, because, besides the outputs of the code you are trying out, descriptive text can help elaborate your ideas better, and even include figures or links in the same file as the code you are working on.
- sharing computational work: Your collaborator/supervisor will be able to follow your reasoning with the descriptive text and check your work in the code blocks and its outputs.

See the complete tutorial here.

6.2 Excel-based orga

Anne has a very nice Excel sheet that she uses as a calendar. It has an annual sheet, a weekly schedule, and more. What is nice about this is that it is not

restricted to Outlook has multiple options for sorting, filtering, coloring, and is completely customizable. Her template is available upon request :)

6.3 Pomodoro timer

"The Pomodoro Technique is a time management method developed by Francesco Cirillo in the late 1980s. It uses a timer to break work into intervals, traditionally 25 minutes in length, separated by short breaks. Each interval is known as a pomodoro, from the Italian word for 'tomato', after the tomato-shaped kitchen timer Cirillo used as a university student." (From wikipedia)

Using the Pomodoro technique can be helpful if you struggle with feeling overwhelmed by your workload. Just start with any task and work on it for the set time (usually 25 minutes), then take a (5 minute) break. You will be surprised by how much you can get done in just one Pomodoro session!

You can either set your phone alarm at your chosen intervals, or you can look up one of the many Youtube channels that provide you with some background white noise during work periods, and music during the breaks. Jana likes to listen to this ambience channel, for example.

6.4 Asana

Basically just on online kanban board. + Its has unlimited projects and a nice look - proprietary and web based.

6.5 Finishing a PhD

Anne collected (with Selina, FU Berlin) all there strategies that helped them to finish their thesis: Tips&Strategies

Chapter 7

Writing & Publishing

7.1 Tools

7.1.1 latex

Latex is a software to write text. You can find thousands of tutorials online. If you want to work with latex on shared documents, consider overleaf.

7.1.2 Others

Other tools can be:

- Markdown
- Word
- Google drive (for collaboration) …

7.2 Writing Papers

Here some general tips for structuring and writing a manuscript.

7.2.1 Structuring the manuscript

- Find a journal of interest, check the author guidelines and follow the instructions. A list of journals of interest can be found in the internal folder of the github.

- The manuscript needs to have page and line numbers, double spaced lines
- **Abstract:**
  - Normally abstracts are around 300 words. For a better abstract, structure it in small sections, much like manuscript itself: Introduction, Methods, Results, Discussion and conclusions.
  - No references in the abstract
- **Introduction:**
  - A ideal introduction has a simple structure without subheaders, but with generally 4-5 paragraphs:
    * First paragraph: Present the key challenge of the manuscript, general importance of this challenge for science and open questions
    * Second paragraph: Details on what people has already found out about the main challenge.
    * Third paragraph: what is the best study system, why and open questions
    * Fourth paragraph: what is the best tool or method strategy and why. This paragraph may be merged with the third one if study system and tool are too intertwined
    * Fifth paragraph: Aims, specific study questions and hypotheses, experimental design, expectations. It will help the paper structure if you explicitly list with numbers very specific questions or hypotheses to be addressed by the experiment and analyses.
  - General suggestion: every paragraph must have the first sentence presenting the topic of the paragraph and everything related to it must be in the same paragraph. If the paragraph is in the introduction, the last sentence may be related to an open question that will be targeted in your study. If the paragraph is in the discussion, the last sentence must be a strong concluding sentence with a take-home message based in your findings.
- **Methods**
  - must have one general, but small paragraph telling what was the main methodological approach.
  - Then the methods must be structured in subsections. Typically, these are:
    * ‚Study system' or ‚empirical data' if empirical or ‚Model description' is modelling manuscript (with many sub-subsections):
      · For modeling papers, follow the ODD protocol [Grimm et al., 2010]

        - It is also useful to document model development, mostly as Appendix, with a TRACE document [Grimm et al., 2014]
    - ‚Experimental design': this must explain what experiments were carrying out to answer i) all the questions; ii) particular questions. Normally you have one figure illustrating the scenarios.
    - ‚Analyses': Start with a general paragraph explaining what model output you used for analyses, whether you did further calculations and manipulations for analyses that are relevant for all study questions. Thereafter, then follow the structure of the hypotheses and create one paragraph for study question and explain what analyses were carried out to answer explicitly each specific study question.

- **Results:**
    - Start explaining general overall findings, then structure according to the study questions/hypotheses. Normally you have one Figure for each question.

- **Discussion:**
    - This is where you discuss the findings in relation to one another and tot he literature. It is useful to start discussing the results in a section per study question. Then add a section on implications to theory and conservation. Another section to limitations and perspectives. Finally a conclusion.

7.2.2 Language Tips

- ‚in order' can be always excluded to save words
- Avoid synonyms, define terms at first mention, preferentially in the introduction and stick to that term throughout the text! You can add a couple of synonyms early in the introduction to increase findability of the manuscript in search engines when it gets publish. However, for most part of the paper use only one term for a given concept.
- You need to edit each sentence. This make sentences simple and direct. Prefer only one verb per sentence.
- You cannot use the conjunction ‚On the other hand," without having used ‚On the one hand,' right before
- In written English, the sentences should always have a noun!!!
    - Example: Wrong: „For plants, some traits do this, others do that." Correct: „For plants, some traits do this, other traits do that."

- In this regard, in any given sentence, there should be only one subject to avoid confusion with the verbs.
  - Example: Wrong: „Plants have traits and some do this." (some plants or some traits?) Correct: „Plants have traits. Some traits do this."

7.3 Rebuttal Letters

7.4 Reviewing Papers

A template of a review feedback letter:

Comments to MS-ID

For editor [This section should be cut and pasted onto the part of the review form online that is confidential to the editor, i.e. not visible to the authors, and should NOT be retained in this file if you decide to upload the comments as a file.]:

[here you talk in brief your general opinion on the paper and justify your recommendation, while mentioning in the aspects, like statistics, that you cannot comment with much confidence. Stress what you think from all your concerns listed below, which are the most critical, those that the authors MUST clarify]

For authors [From this section to the end of the file you can upload to the online review form, normally only necessary when the comments are too extensive.]:

[Here you explain in 1-3 sentences what the paper does and finds (important for the editor). Explain your general assessment on the language and style. Whether the authors can substantiate their claims, whether you see potential in the manuscript, that you may have concerns and what is your general recommendation. Key recommendations are: reject (even if the authors tackle the issues, the manuscript might not fit the scope of the journal, or would remain too trivial and non-original); reject and invite for resubmission (the manuscript has a potential for a good contribution to the field if the major points are correctly addressed, particularly if major points involve re-doing analyses, which mean potentially changed content after revision); major revision (when there are major points, but the authors just need to do some re-writing and/or re-analysing); minor revision (when there is no major point, just minor corrections); accept (everything is at contempt). Be always polite and encouraging.]

Major points:

[Here you list the points that you find critical, which may implicit mean the authors will need to 1) rewrite; 2) re-analyse; 3) re-do experiments. Critical points:

big flaws in the rationale; non-adequate experiment or analyses; wrong conclusions; missing analyses; missing key literature; poor structure. These points can be general and not necessarily addressed to a particular line or sentence. Be always polite and encouraging; if you have any critic, you must provide a solution or a suggestion. Do not criticize just for the sake of it.]

Minor points:

[These are points that you can locate specifically to a line, sentence, paragraph, table or figure. Normally these are unclear statements, typos, errors, missing information, poor design, formatting issues. These can be normally quickly solved.]

Bibliography

Volker Grimm, Uta Berger, Donald L. DeAngelis, J. Gary Polhill, Jarl Giske, and Steven F. Railsback. The odd protocol: A review and first update. *Ecological Modelling*, 221(23):2760–2768, 11 2010. doi: 10.1016/j.ecolmodel.2010.08.019. URL http://linkinghub.elsevier.com/retrieve/pii/S030438001000414X.

Volker Grimm, Jacqueline Augusiak, Andreas Focks, Béatrice M. Frank, Faten Gabsi, Alice S.A. Johnston, Chun Liu, Benjamin T. Martin, Mattia Meli, Viktoriia Radchuk, Pernille Thorbek, and Steven F. Railsback. Towards better modelling and decision support: Documenting model development, testing, and analysis using trace. *Ecological Modelling*, 280:129–139, 05 2014. doi: 10.1016/j.ecolmodel.2014.01.018. URL https://linkinghub.elsevier.com/retrieve/pii/S0304380014000611.

Tianxiao Hao, Jane Elith, Gurutzeta Guillera-Arroita, and José J. Lahoz-Monfort. A review of evidence about use and performance of species distribution modelling ensembles like biomod. *Diversity and Distributions*, 25 (5):839–852, 01 2019. doi: 10.1111/ddi.12892. URL http://dx.doi.org/10.1111/ddi.12892.

Stephanie Kramer-Schadt, Jürgen Niedballa, John D. Pilgrim, Boris Schröder, Jana Lindenborn, Vanessa Reinfelder, Milena Stillfried, Ilja Heckmann, Anne K. Scharf, Dave M. Augeri, Susan M. Cheyne, Andrew J. Hearn, Joanna Ross, David W. Macdonald, John Mathai, James Eaton, Andrew J. Marshall, Gono Semiadi, Rustam Rustam, Henry Bernard, Raymond Alfred, Hiromitsu Samejima, J. W. Duckworth, Christine Breitenmoser-Wuersten, Jerrold L. Belant, Heribert Hofer, and Andreas Wilting. The importance of correcting for sampling bias in maxent species distribution models. *Diversity and Distributions*, 19(11):1366–1379, 07 2013. doi: 10.1111/ddi.12096. URL http://dx.doi.org/10.1111/ddi.12096.

Boris Leroy, Christine N. Meynard, Céline Bellard, and Franck Courchamp. virtualspecies, an r package to generate virtual species distributions. *Ecography*, 39(6):599–607, 06 2015. doi: 10.1111/ecog.01388. URL http://dx.doi.org/10.1111/ecog.01388.

Andrea Saltelli, Stefano Tarantola, Francesca Campolongo, and Marco Ratto. Sensitivity analysis in practice. 02 2002. doi: 10.1002/0470870958. URL http://dx.doi.org/10.1002/0470870958.

Egbert H. van Nes, Marten Scheffer, Marcel S. van den Berg, and Hugo Coops. Charisma: a spatial explicit simulation model of submerged macrophytes. *Ecological Modelling*, 159(2):103–116, 01 2003. doi: 10.1016/S0304-3800(02)00275-2. URL http://www.sciencedirect.com/science/article/pii/S0304380002002752.

Daniel Vedder, Markus Ankenbrand, and Juliano Sarmento Cabral. Dealing with software complexity in individual-based models. *Methods in Ecology and Evolution*, 12(12):2324–2333, 09 2021. doi: 10.1111/2041-210x.13716. URL http://dx.doi.org/10.1111/2041-210X.13716.

Ian J. Wright, Peter B. Reich, Johannes H. C. Cornelissen, Daniel S. Falster, Eric Garnier, Kouki Hikosaka, Byron B. Lamont, William Lee, Jacek Oleksyn, Noriyuki Osada, Hendrik Poorter, Rafael Villar, David I. Warton, and Mark Westoby. Assessing the generality of global leaf trait relationships. *New Phytologist*, 166(2):485–496, 02 2005. doi: 10.1111/j.1469-8137.2005.01349.x. URL http://dx.doi.org/10.1111/j.1469-8137.2005.01349.x.

Damaris Zurell, Janet Franklin, Christian König, Phil J. Bouchet, Carsten F. Dormann, Jane Elith, Guillermo Fandos, Xiao Feng, Gurutzeta Guillera-Arroita, Antoine Guisan, José J. Lahoz-Monfort, Pedro J. Leitão, Daniel S. Park, A. Townsend Peterson, Giovanni Rapacciuolo, Dirk R. Schmatz, Boris Schröder, Josep M. Serra-Diaz, Wilfried Thuiller, Katherine L. Yates, Niklaus E. Zimmermann, and Cory Merow. A standard protocol for reporting species distribution models. *Ecography*, 43(9):1261–1277, 06 2020. doi: 10.1111/ecog.04960. URL http://dx.doi.org/10.1111/ecog.04960.